

# TRAE 基于渐进式索引实现业务上下文管理

本文作者：吴思成，TRAE技术专家

## 一、为什么需要业务上下文管理？

在复杂的软件项目中，代码本身往往无法完整地承载全部业务逻辑。许多关键的业务语义、约束和决策依据，分散在代码之外的各个环节，形成信息鸿沟。若不能有效管理这些“业务上下文”，AI Agent 在理解和开发需求时便会面临巨大挑战。

主要痛点体现在以下几个方面：

- **代码无法完全表达业务语义**：很多业务规则并非通过显式的代码逻辑实现，而是由配置驱动。例如，一个模型的行为模式（如 `__max`、`__v2`）可能由 TCC (Tiktok Configuration Center) 中的某个字段值决定，这种隐式关联对于只读代码的 Agent 来说，几乎无法自行发掘。Agent 可能会因此“重新发明”一套错误的实现，因为它不知道现有逻辑的存在。
- **从 PRD 到 TRD 的信息鸿沟**：产品需求文档（PRD）关注的是“做什么”，而技术实现文档（TRD）关注的是“怎么做”。两者之间存在巨大的认知差距。RD 同学需要结合自身对系统架构、历史包袱、技术栈的理解，才能将 PRD 转化为可执行的技术方案。如果缺少这部分“默认知识”的输入，Agent 同样难以独立完成从需求到方案的转换。
- **隐性知识的缺失**：团队内部存在大量约定俗成的规则、架构边界、关键模块的职责划分等隐性知识。这些信息通常不会在每次需求开发时都明确重申，但却是保证协作顺畅和代码质量的关键。

因此，要让 TRAE 更高效、更准确地参与到方案设计与开发中，就必须为它提供一种能够系统化、结构化地理解和运用这些业务上下文的机制。

## 二、为什么要引入渐进式索引？

在认识到提供业务上下文的重要性后，一个直接的想法是将所有相关的文档（如 Wiki、TRD、架构图等）全部喂给 Agent。这种“一把梭”的策略虽然简单粗暴，但很快就会遇到瓶颈：

- **上下文窗口限制**：大语言模型的上下文窗口是有限的。随着项目文档的不断膨胀，一次性加载所有内容会导致 Token 超限，无法执行任务。
- **高昂的成本与延迟**：即便没有超限，海量的无关信息也会显著增加 Token 消耗，导致更高的成本和更长的响应时间，降低开发效率。
- **信息噪音干扰**：过多的信息会形成噪音，干扰 Agent 的判断。模型需要在大量不相关的内容中大海捞针，反而可能抓不住核心要点，导致方案偏离预期。

为了解决这些问题，我们借鉴了 Anthropic Skills 的“渐进式披露”（Progressive Disclosure）思想，设计并实现了渐进式索引（Progressive Indexing）机制。其核心理念是“按需加载、分层读取”，避免一次性将所有信息灌入上下文，而是引导 Agent 先从一个轻量的“目录”开始，根据当前任务的需要，精准地定位并读取最相关的信息。

这种方式既保证了 Agent 能获取到必要的业务知识，又最大限度地节约了上下文资源，实现了效率与效果的平衡。

### 三、如何生成与组织索引？

渐进式索引的实现依赖于一套标准化的文档结构和规则，它将业务上下文的组织分为两个层次：**元数据（Metadata）** 和 **正文内容（Content）**。

#### 1. 两层结构：Metadata + Content

每一份独立的业务逻辑文档（例如，`model-config-context.md`）都由两部分组成：

- **第一层：元数据（YAML Frontmatter）**
  - **内容：**位于 Markdown 文件头部的 YAML 片段，仅包含两个核心字段：`name` 和 `description`。
    - `name`：业务逻辑的唯一标识符，采用小写字母、数字和连字符的组合（`hyphen-case`）。
    - `description`：对该文档核心内容和适用场景的简要描述，帮助 Agent 判断何时需要读取它。
  - **优势：**元数据极其轻量，加载成本极低。Agent 在启动时可以预加载所有业务文档的元数据，从而对项目拥有的“知识库”有一个全局的概览。
- **第二层：正文内容（Markdown Body）**
  - **内容：**YAML Frontmatter 之后的主体部分，用 Markdown 详细阐述具体的业务逻辑、代码位置、设计决策、注意事项等。
  - **优势：**这部分内容只有在 Agent 根据用户请求和元数据判断其相关时，才会被**按需读取**。这极大地降低了单次请求的 Token 负担。

下面是一个极简的业务逻辑文档示例：

代码块

```
1 ---
2 name: model-version-modes
3 description: "Defines how server matches IDE versionCode and plugin
  versionRule to select effective app/function configs; read when adding version-
  gated configs or debugging config not taking effect."
4 ---
```

```

5
6 # v2 与 Max 模型判定规则
7
8 ## 判定目标
9 - 给定 `IDEFunctionConfig` 或其 `DetailConfigItem`，确定其是否为 v2 模型或 Max 模型
10 - 用于前后端一致的展示与行为控制（提示词、上下文长度、交互流程）
11
12 ## v2 判定信号
13 - 模型层（任一满足即判定为 v2）：
14   - `DetailConfigItem.ModelName` 包含 `__v2`
15
16 ## Max 判定信号
17 - 展示层：
18   - `IDEFunctionConfig.DisplayConfig.MaxMode` == `true`
19 - 模型层（任一满足即判定为 Max）：
20   - `DetailConfigItem.ModelName` 包含 `__max`
21

```

## 2. 作为索引清单的 `project_rules.md`

为了让 Agent 能够发现并使用这些业务文档，我们在项目的 `.TRAE/project_rules.md` 文件中引入了一个特殊的XML块：`<available_business_logic>`。

这个XML块充当了所有可用业务文档的**索引清单**。它由多个 `<business_logic>` 条目组成，每个条目都包含了从对应业务文档 YAML Frontmatter 中提取的 `name`、`description` 等元数据，以及一个指向该文档物理位置的 `<location>` 标签。

代码块

```

1 <!-- .TRAE/project_rules.md -->
2
3 <!-- BUSINESS_LOGIC_INSTRUCTIONS:START -->
4 <business_logic_instructions>
5 When you need to understand a project, you must first check whether there are
6 any
7 available business-logic documents listed below that can help you understand
8 the project more efficiently.
9 ...
10 How to read business-logic:
11 - When you want to read a business-logic document, use the read tool with the
12   parameter set to the <location> of the business-logic file
13 ...
14 Important:
15 - Only read business-logic documents listed in <available_business_logic>
16 - Do not re-read business-logic documents that have already been read
17 </business_logic_instructions>

```

```

17 <!-- BUSINESS_LOGIC_INSTRUCTIONS:END -->
18
19 <available_business_logic>
20   <business_logic>
21     <name>model-config-context</name>
22     <description>
23       Defines the IDECopilot model config context: how TCC maps to
runtime...
24     </description>
25     <location>
26       {{workdir}}/.TRAE/business-logic/model-config-context.md
27     </location>
28   </business_logic>
29   <business_logic>
30     <name>model-version-modes</name>
31     <description>
32       Defines how server matches IDE versionCode and plugin versionRule...
33     </description>
34     <location>
35       {{workdir}}/.TRAE/business-logic/model-version-modes.md
36     </location>
37   </business_logic>
38   ...
39 </available_business_logic>
40

```

**维护策略：**这个索引清单可以通过脚本自动生成和更新。脚本会遍历 `.TRAE/business-logic/` 目录下的所有 `.md` 文件，解析其 YAML Frontmatter，并将元数据和文件路径整合后，更新到 `project_rules.md` 的 `<available_business_logic>` 块中。

通过这种方式，我们实现了业务上下文管理的自动化与标准化。开发者只需按照规范创建或更新业务逻辑文档，索引清单便能自动保持同步。

脚本：



gen\_business\_logic\_index.py  
6.63KB



## 四、如何在 TRAE 中使用业务上下文？

在具备了渐进式索引机制后，我们可以在日常的方案设计、TRD 生成等任务中引导 TRAE 高效地利用这些上下文。Builder 模式由于其强大的指令遵循能力，非常适合这种场景。

以下是在 TRAE 中使用业务上下文的典型流程：

- 1. 启动与加载索引：**当 TRAE Agent 启动处理一个新请求时，它会首先加载 `.TRAE/project_rules.md` 的内容。此时，`<available_business_logic>` 块中所有业务文档的元数据（`name` 和 `description`）便进入了 Agent 的上下文。这给了它一个关于“项目里有哪些已知业务逻辑”的全局视野。
- 2. 需求分析与索引匹配：**当用户提出一个需求（例如，“我需要在 tob 场景下支持 max 模式计费”）时，Agent 会分析需求中的关键词（如“tob”、“max 模式”、“计费”），并将其与内存中所有业务文档的 `description` 进行比对。
  - 如果发现某篇文档的描述（如 `tob-fee-chain` 的描述中包含“ToB streaming billing pipeline”）与需求高度相关，Agent 就会将其标记为待读文档。
- 3. 按需读取详细内容：**在确定了相关的业务文档后，Agent 会使用 `read` 工具，根据索引清单中的 `<location>` 路径，读取这些文档的**完整内容**。此时，详细的业务逻辑、代码位置、注意事项等才真正进入上下文。
- 4. 结合上下文生成方案：**在获得了充分的业务知识后，Agent 便可以开始生成技术方案（TRD）。由于它已经理解了诸如“如何判断 Max 模式”、“ToB 计费链路的入口在哪里”等关键信息，生成的方案会更加贴近实际，避免凭空臆测。
- 5. 引用与校验：**在 TRD 中，Agent 会明确引用它所参考的业务文档，并基于文档中的信息进行设计。例如，它会指出需要修改的具体文件和代码位置，或者建议在哪个配置结构中添加新字段，而不是给出一个模糊的、不具可操作性的方案。

通过这套流程，TRAE 不再是一个盲目读代码的工具，而是一个能够主动查阅、理解和运用沉淀知识的智能助手，从而在复杂的项目中发挥出更大的价值。

更多信息请关注TRAE企业版服务号👉



微信搜一搜



TRAE企业版